

Comparison of Floating Point Values

<https://www.learncpp.com/cpp-tutorial/relational-operators-and-floating-point-comparisons/>

Comparing Floating Point Values

```
1  #include <iostream>
2
3  int main()
4  {
5      double d1{ 100.0 - 99.99 }; // should equal 0.01 mathematically
6      double d2{ 10.0 - 9.99 }; // should equal 0.01 mathematically
7
8      if (d1 == d2)
9          std::cout << "d1 == d2" << '\n';
10     else if (d1 > d2)
11         std::cout << "d1 > d2" << '\n';
12     else if (d1 < d2)
13         std::cout << "d1 < d2" << '\n';
14
15     return 0;
16 }
```

*We talked about how
comparing two floating
point values is unreliable.*

Variables d1 and d2 should both have value 0.01. But this program prints an unexpected result:

```
d1 > d2
```

Comparing Floating Point Values

- This situation may come up during your lab.
- Here is a way to solve that problem.

Comparing Floating Point Values

- Let's say you have two floating point values,

$$a = 10.00\underline{101}$$

$$b = 10.00\underline{092}$$

- Whether these numbers are the **same or not**, might depend on the **context** in which they're being used.
 - If these are **lengths of two pencils**, we would probably say that they are the same.
 - If these are **lengths of two mechanical parts** that fits into a **spacecraft or a Formula-1 racecar**, we might think otherwise.

Comparing Floating Point Values

a	=	10.00 <u>101</u>
b	=	10.00 <u>092</u>

- The point is that the acceptable precision can vary.
- When comparing floating point values, you can make use of this concept.
- Let's say you are comparing the **lengths of two pencils**, you can set your precision to be **0.01**.
 - In this e.g., both numbers become the same.

So let's try to create a function for this.

```
bool approxEqual(double a, double b, double prec)
{
    return std::abs(a - b) <= prec;
}
```

```
a = 10.00101
b = 10.00092
prec = 0.01
```

You'll need,
#include<cmath>
for using `std::abs()`

Comparing Floating Point Values

- The precision is generally called “**epsilon**”.
- Following shows more details in comments.
 - Functionally, what I showed before, does the same thing.

```
1 | #include <cmath> // for std::abs()
2 |
3 | // absEpsilon is an absolute value
4 | bool approximatelyEqualAbs(double a, double b, double absEpsilon)
5 | {
6 |     // if the distance between a and b is less than or equal to absEpsilon, then a and b are "close enough"
7 |     return std::abs(a - b) <= absEpsilon;
8 | }
```